

// JavaJub

Собес в VK

Группа балансеров • One-cloud

Java Middle

Вопросы, задачи и подготовка
к live-coding и техническому интервью

Стек

Java 21 • Spring Boot 3 • Kafka • PostgreSQL • Kubernetes • gRPC • HAProxy • Docker

[// содержание](#)

Содержание

01	Про VK и группу балансеров	3
02	Стек по вакансии	5
03	Java Core — что точно спросят	6
04	Коллекции	9
05	Многопоточность и JMM	10
06	Spring и Spring Boot	12
07	Базы данных и SQL	13
08	Kafka и микросервисы	15
09	Сети и балансировка	16
10	Docker, Kubernetes, CI/CD	17
11	Практические задачи (live-coding)	18
12	System Design	21
13	План подготовки + чек-лист	22

01 Про VK и группу балансеров

VK — одна из крупнейших IT-компаний России (15 000+ сотрудников, 200+ продуктов, 95% аудитории рунета). Центр Технологий VK отвечает за IT-инфраструктуру всех продуктов: ВКонтакте, ОК, Дзен, Mail.ru, RuStore, VK Play.

One-cloud — технологический фундамент VK: единая среда запуска приложений, хранилищ, баз данных и сервисов. Группа балансеров занимается управлением и администрированием L3- и L4-балансеров облака. Через них идёт ВЕСЬ сервисный и пользовательский трафик — это tier 1 уровень стабильности.

► Чем занимается группа балансеров

- Разработка Java-бэкенда инфраструктурного сервиса для регулирования трафика в дата-центрах
- Написание Java-клиентов для взаимодействия с другими инфраструктурными сервисами
- Создание API с авторизацией для взаимодействия пользователей через UI
- Управление L3- и L4-балансеров: чтобы другие разработчики не думали о тонкостях балансировки
- Поддержание высочайшего уровня стабильности (tier 1) — любой сбой = падение всех продуктов VK

► Как устроен процесс найма

По отзывам кандидатов (DreamJob, Habr 879068, Habr 1006022): VK проводит единое техническое интервью для всех Java-вакансий, после которого распределяет по командам. Процесс похож на Яндекс, но обычно быстрее.

Этап	Формат	Длительность
HR-скрининг	Zoom/Telegram, мотивация, ЗП, стек	20–30 мин
Техническое интервью	Теория + live-coding + SQL	60–90 мин
Алгоритмическая секция	Live-coding, 2–3 задачи	45–60 мин
Архитектурная секция	System Design, Miro + Zoom	45–60 мин
Финал с командой	Знакомство с тимлидом и командой	30–60 мин

ФИШКА • Единое интервью

В VK техническое интервью общее для всех Java-вакансий. HR предлагает 2–3 команды на выбор, но тех-собес один. Это значит: готовиться нужно к общей Java-базе, а не к специфике балансеров. Специфика обсуждается на финале с командой.

ВНИМАНИЕ • Что бросается в глаза

По отзывам на Habr: VK спрашивает алгоритмы и структуры данных даже у Middle. Бинарный поиск, хэш-таблицы, деревья, очереди с приоритетом — будьте готовы. На этапе SQL всё обычно проходит хорошо, а вот алгоритмы — главный фильтр.

02 Стек по вакансии

Группа балансеров — часть One-cloud, разработка ведётся на современном стеке без устаревшего кода.

► Основной стек

- Java 21 — последняя LTS-версия, Virtual Threads, Records, Pattern Matching
- Spring / Spring Boot 3 — основной фреймворк
- Kafka — брокер сообщений для межсервисного взаимодействия
- PostgreSQL — основная СУБД
- Kubernetes — оркестрация контейнеров (One-cloud построен поверх K8s)
- Docker / Docker Compose — контейнеризация
- gRPC — внутренний RPC между инфраструктурными сервисами
- Prometheus + Grafana — мониторинг и метрики
- HAProxy / Nginx / Envoy — L3/L4/L7 балансеры (предметная область команды)

► Будет плюсом

- Понимание сетевых протоколов: TCP/IP, UDP, HTTP/2, VRRP
- Опыт с L3/L4/L7 балансировкой (HAProxy, Nginx, Envoy, IPVS)
- Опыт работы с OpenStack, облачными платформами
- Ant, Gradle — системы сборки (указаны в вакансии)
- Навыки автоматизации тестирования
- Опыт разработки и интеграции сложных внутренних продуктов

СОВЕТ · Про Java 21

VK использует Java 21 — это важно. Готовьтесь к вопросам про Virtual Threads (Project Loom), Records, Sealed Classes, Pattern Matching for switch. Это не «будет плюсом», а рабочий стек.

03 Java Core — что точно спросят

`equals/hashCode`, `HashMap`, `Stream API`, многопоточность — must-have. VK спрашивает глубже среднего: устройство JVM, GC, ссылочные типы.

► Базовые вопросы

- JDK, JRE, JVM.
 - JVM — виртуальная машина (исполняет байт-код). JRE = JVM + библиотеки. JDK = JRE + инструменты. С Java 9 JRE отдельно не поставляется. Java 21: *Virtual Threads, Records, Sealed Classes, Pattern Matching*.
- Области памяти JVM.
 - *Heap (Eden/Survivor/Old), Stack (фреймы), Metaspace (метаданные классов, раньше PermGen), PC Register, Native Method Stack, CodeCache (JIT)*. GC работает только с *Heap*.
- Контракт `equals/hashCode`.
 - *`a.equals(b)` → `hashCode` одинаков. Обратное необязательно. Переопределяешь один — переопределяй оба. Нарушение ломает `HashMap/HashSet`: объект «теряется» при изменении поля в `hashCode`.*
- Что сломается, если `hashCode()` — константа?
 - *Все в одном bucket. Java 8+: при 8 коллизиях И `capacity ≥ 64` → *red-black tree* $O(\log n)$ вместо $O(1)$.*
- String Pool и immutability.
 - *Pool в heap — уникальные литералы. `new String()` — вне пула. `intern()` добавляет. Immutability: безопасность, потокобезопасность, кэширование `hashCode`, пул.*
- Generics: type erasure, PECS.
 - *Type erasure: в рантайме нет типового параметра. PECS: Producer Extends (читать), Consumer Super (писать). Wildcard `?` — только чтение как `Object`.*
- WeakReference / SoftReference / PhantomReference.
 - *Weak — GC собирает при первой сборке (`WeakHashMap`). Soft — при нехватке памяти (кэши). Phantom — постфинализация очистка. `ReferenceQueue` для уведомлений.*
- Функциональные интерфейсы.
 - *Один абстрактный метод. `Function<T,R>`, `Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `UnaryOperator<T>`, `BiFunction`, `Comparator`, `Runnable`, `Callable`.*
- Stream API.
 - *Промежуточные (ленивые): `filter`, `map`, `flatMap`, `sorted`, `distinct`. Терминальные: `collect`, `forEach`, `count`, `reduce`, `toList`. Обработка вертикальная. Стрим одноразовый.*
- Virtual Threads (Java 21).
 - *Лёгкие потоки, управляемые JVM (не ОС). `Thread.ofVirtual().start()`. Миллионы потоков без проблем. НЕ подходят для CPU-bound задач (нет `preemption`). Идеальны для I/O-bound (сетевые запросы, БД). `Executors.newVirtualThreadPerTaskExecutor()`.*
- Records (Java 16+).
 - *Иммутабельные data-классы: `record Point(int x, int y) {}`. Автоматически: конструктор, `getters`, `equals`, `hashCode`, `toString`. Нельзя наследовать (`implicitly final`). Можно реализовывать интерфейсы.*
- Sealed Classes (Java 17+).
 - *Ограничение иерархии наследования: `sealed class Shape permits Circle, Square`. Компилятор гарантирует `exhaustive switch`. Связка с `Pattern Matching for switch` (Java 21).*
- `final`: класс, метод, поле.

→ Класс — нельзя наследовать. Метод — нельзя переопределить. Поле — нельзя переприсвоить (мутабельное содержимое можно менять). *effectively final* — для лямбд.

► Задачи «Что выведет?»

Тест 1. Integer cache

```
Integer a = 127, b = 127;
System.out.println(a == b); // true

Integer c = 128, d = 128;
System.out.println(c == d); // false
```

IntegerCache: -128..127. Для 127 — один объект. Для 128 — два разных. Мораль: для объектов — equals().

Тест 2. String Pool

```
String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello");

System.out.println(s1 == s2);    // true  (оба из пула)
System.out.println(s1 == s3);    // false (s3 — новый объект)
System.out.println(s1.equals(s3)); // true  (содержимое одинаковое)
```

Тест 3. Stream — ленивость

```
List.of(1,2,3,4,5).stream()
    .peek(x -> System.out.print("A" + x + " "))
    .filter(x -> x % 2 == 0)
    .peek(x -> System.out.print("B" + x + " "))
    .toList();
// A1 A2 B2 A3 A4 B4 A5
```

Вертикальная обработка: каждый элемент проходит всю цепочку. Нечётные не проходят filter → B не печатается.

СОВЕТ • Про Java 21

VK использует Java 21. Готовьтесь к вопросам про Virtual Threads, Records, Sealed Classes, Pattern Matching for switch. Пример: «Чем Virtual Thread отличается от Platform Thread? Когда НЕ стоит использовать Virtual Threads?» — CPU-bound задачи, synchronized блоки (pinning).

04 Коллекции

HashMap — абсолютный чемпион вопросов. В VK спрашивают глубоко: treeify threshold, resize, null-ключи, ConcurrentHashMap внутренности.

- HashMap — устройство.
 - *Node<K,V>[]*. Размер — степень двойки (default 16). Индекс: $(n-1) \& \text{hash}(\text{key})$. Коллизии — список. Java 8+: TREEIFY_THRESHOLD=8 И $\text{capacity} \geq 64 \rightarrow \text{red-black tree}$.
- load factor и resize.
 - 0.75 по умолчанию. $\text{size} \geq \text{capacity} * \text{loadFactor} \rightarrow \text{resize}$ (вдвое + перехеширование всех). Совет: $\text{initialCapacity} = \text{expectedSize} / 0.75 + 1$.
- HashMap vs ConcurrentHashMap.
 - HashMap: не потокобезопасен, null-ключ. ConcurrentHashMap: Java 8+ CAS + synchronized на головках бакетов. null запрещён. computeIfAbsent атомарен.
- ArrayList vs LinkedList.
 - ArrayList: $O(1)$ доступ, CPU-кэш. LinkedList: $O(1)$ вставка в начало. На практике ArrayList всегда лучше.
- TreeMap vs LinkedHashMap.
 - TreeMap: red-black tree, $O(\log n)$, отсортирован. LinkedHashMap: порядок вставки / `accessOrder=true` для LRU.
- PriorityQueue.
 - Min-heap по умолчанию. $O(\log n)$ offer/poll, $O(1)$ peek. Для Top-K задач. Comparator для кастомного порядка.
- Почему Tree индекс в БД, а не Hash?
 - Хотя $\text{Hash} = O(1)$, Tree (B-tree) поддерживает диапазонные запросы, сортировку, BETWEEN, LIKE 'abc%'. Hash — только точное совпадение.

05 Многопоточность и JMM

В VK многопоточность критична — балансеры обрабатывают весь трафик. `volatile` vs `synchronized` спрашивают на каждом собеседе. `ThreadPoolExecutor`, CAS, пулы потоков — обязательно.

- `synchronized`.
→ *Захват монитора. Instance → this. Static → Class. Reentrant (счётчик). Mutual exclusion + happens-before.*
- `volatile`.
→ *Видимость + запрет reordering. НЕ атомарность i++. Для атомарных — AtomicInteger/LongAdder.*
- `happens-before`.
→ *unlock → lock. volatile write → read. Thread.start() → первая инструкция. join(). final-поля после конструктора.*
- `ConcurrentHashMap` Java 8.
→ *CAS + synchronized на головах бакетов. Treeify при 8 коллизиях. null запрещён. computeIfAbsent атомарен.*
- `ThreadPoolExecutor`.
→ *corePoolSize, maxPoolSize, keepAliveTime, workQueue, threadFactory, rejectedExecutionHandler. newCachedThreadPool опасен — OOM.*
- `Deadlock`.
→ *Два+ потока ждут друг друга. Условия Коффмана. Решение: упорядочить захват мониторов, tryLock с таймаутом. Обнаружение: jstack.*
- `CompletableFuture`.
→ *thenApply (map), thenCompose (flatMap), thenCombine (join двух). exceptionally. Async-варианты в ForkJoinPool.*
- `Virtual Threads` и `synchronized`.
→ *Virtual Thread на synchronized → pinning (привязка к platform thread). Решение: ReentrantLock вместо synchronized. Это частый вопрос для Java 21.*

ЛОВУШКА • `volatile counter++`

`volatile long counter; counter++` — НЕБЕРНО (три операции). Три корректных варианта: `synchronized`, `AtomicLong.incrementAndGet()`, `LongAdder.increment()`. `LongAdder` лучше при высоком contention.

06 Spring и Spring Boot

Spring Boot 3 — основной фреймворк VK для Java-сервисов. Спрашивают: прокси, жизненный цикл, автоконфигурация, транзакции.

- IoC и DI.
 - IoC — контейнер управляет. DI — зависимости внедряются. *Constructor injection* лучший: *final*, явные зависимости, тестируемость.
- Жизненный цикл бина.
 - *BeanDefinition* → *инстанцирование* → *DI* → *Aware* → *BPP.before* → *@PostConstruct* → *InitializingBean* → *init-method* → *BPP.after* → *ГОТОВ* → *@PreDestroy* → *destroy*.
- @Transactional.
 - Прокси (JDK/CGLIB). Открывает транзакцию, *commit/rollback*. *self-call* минует прокси. *Rollback* на *RuntimeException/Error*; *checked* — нужен *rollbackFor*.
- Propagation.
 - *REQUIRED* (default), *REQUIRES_NEW*, *NESTED*, *SUPPORTS*, *MANDATORY*, *NOT_SUPPORTED*, *NEVER*.
- @SpringBootApplication.
 - *@Configuration* + *@EnableAutoConfiguration* + *@ComponentScan*.
- Scope: prototype в singleton.
 - *Prototype*-бин создастся один раз при инъекте. Решение: *Provider<T>*, *ObjectFactory<T>*, *@Lookup*.
- Spring Boot 3 + Java 21.
 - *GraalVM native image*, *Virtual Threads support* (*spring.threads.virtual.enabled=true*), *Jakarta EE* вместо *javax*.

07 Базы данных и SQL

PostgreSQL — основная БД в VK для Java-сервисов. SQL спрашивают отдельно: JOIN, GROUP BY, HAVING, индексы, транзакции.

- ACID.
→ *Atomicity, Consistency, Isolation, Durability. WAL в PostgreSQL для Durability.*
- Уровни изоляции.
→ *READ_UNCOMMITTED, READ_COMMITTED (default PG), REPEATABLE_READ, SERIALIZABLE. Аномалии: dirty/non-repeatable/phantom read.*
- Индексы PostgreSQL.
→ *B-tree (default), Hash, GIN (JSONB, full-text), GiST (геометрия), BRIN (большие таблицы). Покрывающий (INCLUDE).*
- Почему B-tree, а не Hash?
→ *Hash: $O(1)$ только =. B-tree: $O(\log n)$ но поддерживает <, >, BETWEEN, ORDER BY, LIKE 'abc%'. Поэтому B-tree — default.*
- Порядок колонок в составном индексе.
→ *Leftmost prefix rule. INDEX (a, b, c) используется для WHERE a=, WHERE a= AND b=, но НЕ для WHERE b= или WHERE c=.*
- EXPLAIN ANALYZE.
→ *Seq Scan, Index Scan, Bitmap Heap Scan. Nested Loop vs Hash Join. Rows Removed by Filter. estimated vs actual rows.*
- Оконные функции.
→ *ROW_NUMBER(), RANK(), LAG/LEAD, SUM OVER (PARTITION BY ... ORDER BY ...). Агрегация без сворачивания строк.*

► SQL-задачи

Вторая по величине зарплата

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Идемпотентное зачисление через Kafka

Из собеседов на Java Middle: поручения на зачисление денег приходят по Kafka. Kafka гарантирует доставку, но НЕ единственность. Как обеспечить, что деньги зачисляются ровно один раз?

```
-- Таблица идемпотентности
CREATE TABLE processed_events (
    event_id UUID PRIMARY KEY,
    processed_at TIMESTAMP DEFAULT NOW()
)
;

-- В транзакции с бизнес-логикой:
INSERT INTO processed_events (event_id)
VALUES (:eventId)
ON CONFLICT (event_id) DO NOTHING;
-- Если вставка прошла — обработать
-- Если конфликт — skip (уже обработано)
```

08 Kafka и микросервисы

Kafka указана в стеке VK для Java-сервисов. Для группы балансеров это межсервисная коммуникация между инфраструктурными компонентами.

- Kafka — зачем?
→ Распределённый лог-ориентированный брокер. Topic → Partition → Offset. Данные не удаляются после прочтения (retention). Асинхронность, буферизация, аудит.
- Consumer Group.
→ Каждая партиция — ровно один consumer из группы. Больше партиций → больше параллелизма. Rebalancing при изменении состава.
- Гарантии доставки.
→ at-most-once, at-least-once (default), exactly-once (idempotent producer + transactional). В проде: at-least-once + идемпотентность consumer.
- Идемпотентный consumer.
→ processed_events(event_id UUID PK). INSERT ON CONFLICT DO NOTHING. В транзакции с бизнес-логикой. Или upsert по бизнес-ключу.
- Circuit Breaker.
→ Защита от каскадных сбоев. CLOSED → OPEN → HALF_OPEN. Resilience4j. fallback-ответ при недоступности downstream.
- Saga-паттерн.
→ Распределённые транзакции через цепочку локальных + компенсации. Choreography vs Orchestration.

09 Сети и балансировка

Группа балансеров — это про сети. Даже если глубокие сетевые вопросы будут на финале с командой, базовое понимание L3/L4/L7 балансировки ожидается.

- L3 vs L4 vs L7 балансировка.
 - L3 (сетевой): IP-маршрутизация, ECMP. L4 (транспортный): TCP/UDP, NAT, по IP:port без разбора содержимого. L7 (прикладной): HTTP-заголовки, URL, cookies. L4 быстрее (меньше overhead), L7 гибче.
- HAProxy vs Nginx vs Envoy.
 - HAProxy: L4/L7, production-proven, простая конфигурация. Nginx: L7, статика + reverse proxy. Envoy: L4/L7, gRPC, service mesh (Istio). В VK — все три в разных сценариях.
- Health checks.
 - Active: балансер периодически проверяет backend (HTTP GET /health, TCP connect). Passive: считает ошибки от реальных запросов. Комбинация — best practice.
- VRRP.
 - Virtual Router Redundancy Protocol. active-standby пара балансеров. При падении primary — мгновенное переключение, IP не меняется. Используется в VK Cloud.
- Sticky sessions.
 - Привязка клиента к конкретному backend. По cookie, IP, header. Проблема: неравномерная нагрузка при скейлинге. Решение: shared session storage (Redis) или stateless (JWT).
- Connection draining.
 - При выводе backend из ротации — дождаться завершения активных соединений, а не обрывать. Graceful shutdown.

ФИШКА • Почему это важно

Через балансеры группы идёт ВСЬ трафик VK — ВКонтакте, ОК, Дзен, Mail.ru. Это tier 1: любой сбой = даунтайм всех продуктов. На финале с командой точно спросят про L3/L4 разницу и как вы думаете о стабильности.

10 Docker, Kubernetes, CI/CD

One-cloud VK построен поверх Kubernetes. Docker и K8s — рабочие инструменты группы балансеров.

- Образ vs контейнер.
→ Образ — неизменяемый шаблон из слоёв. Контейнер — запущенный экземпляр с writable-слоем.
- Multi-stage build.
→ Сборка → финальный образ только JAR. ~200 MB вместо ~800 MB. Безопаснее (меньше attack surface).
- Kubernetes: Pod, Deployment, Service.
→ Pod — 1+ контейнер. Deployment — реплики, rolling update. Service — стабильный endpoint. Ingress — L7 маршрутизация.
- Service типы в K8s.
→ ClusterIP (внутренний), NodePort (внешний на каждой ноде), LoadBalancer (облачный LB — именно это делает группа балансеров в VK).
- Liveness vs Readiness probe.
→ Liveness: жив ли → перезапуск. Readiness: готов ли → убирается из балансировки. Spring Boot Actuator: /health/liveness, /health/readiness.
- ConfigMap vs Secret.
→ ConfigMap — не-секретная конфигурация. Secret — base64-encoded чувствительные данные. Оба монтируются как volumes или env vars.

11 Практические задачи (live-coding)

По отзывам кандидатов в VK: задачи на алгоритмы, Stream API, многопоточность. Пишете код в онлайн-редакторе. Рассуждайте вслух!

► Задача 1. Найти дубликаты

```
public Set<Integer> findDuplicates(List<Integer> list) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dups = new HashSet<>();
    for (Integer n : list) {
        if (!seen.add(n)) dups.add(n);
    }
    return dups;
}
```

$O(n)$ время, $O(n)$ память. `seen.add()` → false если элемент уже есть.

► Задача 2. Являются ли строки перестановками

Из реальных собеседов: определить, являются ли две строки перестановкой символов (a..z) друг друга.

```
public boolean isPermutation(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] counts = new int[26];
    for (char c : a.toCharArray()) counts[c - 'a']++;
    for (char c : b.toCharArray()) counts[c - 'a']--;
    for (int count : counts) {
        if (count != 0) return false;
    }
    return true;
}
```

$O(n)$ через counting sort. Альтернатива: отсортировать обе строки $O(n \log n)$. Counting быстрее для маленького алфавита.

► Задача 3. REST-контроллер

```
@RestController
@RequestMapping("/api/balancers")
public class BalancerController {

    private final BalancerService service;

    public BalancerController(BalancerService service) {
        this.service = service;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public BalancerDto create(@RequestBody @Valid CreateBalancerRequest req) {
        return service.create(req);
    }

    @GetMapping("/{id}")
```

```

    public BalancerDto getById(@PathVariable Long id) {
        return service.findById(id);
    }
}

```

► Задача 4. Потокобезопасный кэш

```

public class SimpleCache<K, V> {
    private final Map<K, V> cache = new ConcurrentHashMap<>();

    public V get(K key, Function<K, V> loader) {
        return cache.computeIfAbsent(key, loader);
    }
}

```

ConcurrentHashMap (не HashMap). computeIfAbsent (не containsKey+put — гонка). Ограничение размера: Caffeine с maxSize и TTL.

► Задача 5. Code Review

```

public class UserCache {
    private static Map<Long, User> cache = new HashMap<>();

    public static User getUser(Long id) {
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        User user = loadFromDb(id);
        cache.put(id, user);
        return user;
    }
}

```

Проблемы: 1) HashMap не потокобезопасен → ConcurrentHashMap. 2) containsKey+get → computeIfAbsent. 3) Кэш бесконечный → memory leak. 4) static → тяжело тестировать. 5) null от loadFromDb.

► Задача 6. SQL: идиempотентное зачисление

Из Пикабу (реальный вопрос на Java Middle): система зачисления денег через Kafka. Как гарантировать, что деньги зачислятся ровно один раз?

```

@Transactional
public void processPayment(PaymentEvent event) {
    try {
        processedEventRepo.save(
            new ProcessedEvent(event.getId()));
    } catch (DataIntegrityViolationException e) {
        log.info("Duplicate event: {}", event.getId());
        return; // уже обработано
    }
    accountRepo.credit(event.getAccountId(), event.getAmount());
}

```

INSERT + бизнес-логика в одной транзакции. PK conflict = дубль = skip. Kafka at-least-once + идиempотентность = exactly-once семантика.

12 System Design

Для Middle архитектурная секция может быть опциональной, но для Middle+ — обязательна. В контексте балансеров: ожидайте задачи про распределённые системы и высокую нагрузку.

- Спроектируй балансировщик нагрузки.
→ Компоненты: *health checker, routing table, connection pool*. Алгоритмы: *round-robin, weighted, least-connections, consistent hashing*. Отказоустойчивость: *active-standby (VRRP), hot reload* конфигурации.
- Сервис коротких ссылок.
→ *base62* ($62^7 \approx 3.5$ трлн). *Redis-кэш* популярных. Шардирование по хэшу. Расчёт: чтения/записи 100:1.
- Распределённый rate limiter.
→ *Token Bucket vs Sliding Window*. *Redis + Lua*. Точность vs производительность.
- Сервис метрик (Prometheus-like).
→ *Time-series DB*. Ingestion: *push (StatsD) vs pull (Prometheus)*. *Downsampling* для старых данных. *Retention policy*.

13 План подготовки + чек-лист

► За 2–3 недели

- 30–40 задач LeetCode Easy+Medium: hash-table, two-pointers, sliding-window, BFS/DFS
- Повторить Java 21: Virtual Threads, Records, Sealed Classes, Pattern Matching
- Spring Boot 3 + PostgreSQL + Kafka — поднять pet-проект
- Повторить HashMap устройство, ConcurrentHashMap, ThreadPoolExecutor
- Почитать про L3/L4 балансировку, HAProxy, Kubernetes Service types

► За неделю

- 2–3 мок-интервью: pramp.com, interviewing.io, друзья
- Все вопросы из гайда ВСЛУХ — мысли в голове ≠ слова
- 2–3 проекта по STAR: проблема → что сделал → результат
- Повторить SQL: JOIN, GROUP BY, HAVING, оконные функции, EXPLAIN
- Почитать про One-cloud VK, роль группы балансеров

► В день собеседа

- Камера, микрофон, интернет — за 30 минут
- Рассуждать вслух — молчание хуже «дай подумать»
- Не знаешь — честно: «не сталкивался, но предположу...»
- 2–3 вопроса в конце: про One-cloud, команду, стек, процессы

ВНИМАНИЕ • Главный фильтр

По отзывам кандидатов VK: алгоритмы и структуры данных — главный фильтр. Хэш-таблицы, деревья, очереди с приоритетом. Это спрашивают даже у Middle. SQL обычно проходит хорошо. Готовьтесь к алгоритмам!

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	equals/hashCode + Integer cache + Virtual Threads (Java 21) + Records
Коллекции	HashMap: treeify, resize, ConcurrentHashMap Java 8, PriorityQueue
Многопоточность	volatile vs synchronized + CAS + ThreadPoolExecutor + Virtual Threads pinning
Spring	@Transactional прокси + self-call + scope prototype в singleton + Spring Boot 3
SQL	JOIN + GROUP BY + составной индекс + оконные функции + EXPLAIN ANALYZE

Kafka	Consumer Group + at-least-once + идемпотентность + exactly-once
Сети	L3 vs L4 vs L7 балансировка + HAProxy + VRRP + health checks
Docker/K8s	Dockerfile multi-stage + Pod/Deployment/Service + probes + Service types
System Design	Load Balancer design + rate limiter + back-of-envelope расчёты
Live-coding	3 задачи за 1 час: перестановки строк, дубликаты, code review

Удачи на собеседе!

// git push origin offer

JavaJub